COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09 - 16 - 26	**DATE SUBMITTED:**	09 - 16 - 26
NAME:	Paula Esguerra	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA NO. _9__ Implementing Trees

1. PROCEDURE OUTPUT

Array of Values for Sort Algorithm Testing

Screenshot

main.cpp

```cpp
#include <iostream>
#include "binaryTree.h"

main(){

    //creating the root node:
    BinaryTree<char>* root = new BinaryTree<char>('A');

    //creating a child of root node A:
    root->insertLeftChild('B');
    root->insertRightChild('C');

    //insert a child to node B;
    root->getLeftChild()->insertLeftChild('D');
    root->getLeftChild()->insertRightChild('F');


    //insert a child to node C;
    root->getRightChild()->insertLeftChild('E');

    //print the root node:
    std::cout << "Root: " << root->getRootVal() << std::endl;

    //children of the root node:
    std::cout << "Root Left Child: " << root->getLeftChild()->getRootVal() << std::endl;
    std::cout << "Root Right Child: " << root->getRightChild()->getRootVal() << std::endl;

    //children of the sub trees B and C
    std::cout << "node B Left Child: " << root->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "node B Right Child: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;

    root->getLeftChild()->getRightChild()->setRootVal('E');
    std::cout << "node B Right Child: " << root->getLeftChild()->getRightChild()->getRootVal() << std::endl;


    return 0;
}
```

binaryTree.h

```
#ifndef BINARYTREE_H
#define BINARYTREE_H

template<typename T>
class BinaryTree{
    private:
        T key;
        //pointers:
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;

    public:

        //constructor:
        BinaryTree(T rootObj);

        //adding/inserting a child node:
        void insertLeftChild(T newNode);
        void insertRightChild(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();
        T getRootVal();
        void setRootVal(T obj);
        bool isLeaf();
};

template<typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key =rootObj;
```

```
31      leftChild = nullptr;
32      rightChild = nullptr;
33  }
34
35  template<typename T>
36  void BinaryTree<T>::insertLeftChild(T newNode){
37      if(leftChild == nullptr){
38          leftChild = new BinaryTree<T>(newNode);
39      }else{
40
41          //create a new node:
42          BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
43
44          //move the existing left child below the new node
45          newChild->leftChild = leftChild;
46          leftChild = newChild;
47      }
48  }
49
50  template <typename T>
51  void BinaryTree<T>::insertRightChild(T newNode) {
52      if(rightChild == nullptr){
53          rightChild = new BinaryTree<T>(newNode);
54      }else{
55          //create a new node:
56          BinaryTree<T> *newChild = new BinaryTree<T>(newNode);
57
58          //move the existing left child below the new node
59          newChild->rightChild = rightChild;
60          rightChild = newChild;
```

```
61      }
62  }
63
64  template<typename T>
65  BinaryTree<T>* BinaryTree<T>::getLeftChild(){
66      return leftChild;
67  }
68
69  template<typename T>
70  BinaryTree<T>* BinaryTree<T>::getRightChild(){
71      return rightChild;
72  }
73
74  template<typename T>
75  T BinaryTree<T>::getRootVal(){
76      return key;
77  }
78
79  template<typename T>
80  void BinaryTree<T>::setRootVal(T obj){
81      key = obj;
82  }
83
84  template<typename T>
85  bool BinaryTree<T>::isLeaf(){
86      return leftChild == nullptr && rightChild == nullptr;
87  }
88  #endif
```

Output

```
C:\Users\Paramount Laptop 5
Root: A
Root Left Child: B
Root Right Child: C
node B Left Child: D
node B Right Child: F
node B Right Child: E

--------------------------------
Process exited after 2.128 seconds with return value 0
Press any key to continue . . .
```

Observations

This implementation constructs a hierarchical tree using char data values to display node relationships. By utilizing template<typename T>, the BinaryTree becomes

completely reusable across different data types like char, int, and custom objects. Its public functions utilize leaf properties to maintain strict encapsulation. Once the missing methods are implemented and proper data fields are fetched, this pointer-based, non-linear linked data structure will successfully navigate the binary tree network.

BinaryTree(T rootObj)
The constructor initializes a brand new tree node.

T getRootVal()
Returns the data is currently held in the private key variable.

void setRootVal(T obj)
It changes the private variable key to match the new value (obj) you passed into the function.

Bool isLeaf()
Checks if the current node is a “leaf”

BinaryTree<T>* getLeftChild() & BinaryTree<T>* getRightChild()
Grant access to the sub-trees connected to the current node.

void insertLeftChild(T newNode)
The new node pushes its ways into the structure, making the original left child its own left child.

void insertRightChild(T newNode)
Operates under the exact same structural shifting rules as insertLeftChild, but applies them exclusively to the right branch pointer (rightChild).

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

3vb . ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.